

Binary Search Algorithm like first the elements should be arranged in ascending order then later the key element will be present key element will be present based on that condition we have to change for the low or the mid value.

Given array:

66 111 212 323 432 543 643 649 777 888
0 1 2 3 4 5 6 7 8 9

i) case 1: 111

ii) case 2: 888

iii) case 3: 999

Time Complexity: $O(1)$ $\Rightarrow$ best case, $O(\log n) \Rightarrow$ Average case, $O(\log n) \Rightarrow$ worst case

space complexity: $O(1)$

i) $\text{low} = 0, \text{High} = 9$

$$\text{Mid} = \frac{\text{L} + \text{H}}{2} = \frac{0 + 9}{2} = 4.5$$

$$A(4) = 432$$

$$111 < 432$$

ii) $\text{low} = 0, \text{High} = 4$

$$\text{Mid} = \frac{0 + 4}{2} = 2$$

$$A(2) = 212$$

Found

Result: Found

Position = 2

Index = 1

Comparisons = 2

② 888

$$\text{low} = 0, \text{High} = 9$$

$$\text{Mid} = \frac{\text{L} + \text{H}}{2} = 4$$

$$432 < 888$$

$$\text{low} = 5$$

$$\text{low} = 5, \text{High} = 9$$

$$\text{Mid} = 7$$

$$649 < 888$$

$$\text{low} = 8$$

$$\text{High} = 9$$

$$\text{Mid} = 8$$

$$777 < 888$$

$$\text{low} = 9$$

$$\text{High} = 9$$

$$\text{Mid} = 9$$

$$888 = 888$$

Found

Position = 8

Comparisons = 4

⑤ 999

$$\text{low} = \frac{0+9}{2}$$

$$M = 4$$

$$432 < 999$$

$$\text{low} = 5$$

$$\text{High} = 9$$

$$\text{Mid} = 7$$

$$649 < 999$$

$$\text{Low} = 8$$

$$\text{High} = 9$$

$$\text{mid} = 8$$

$$777 < 999$$

$$\text{low} = 10$$

$$\text{low} = 9, \text{High} = 9 \Rightarrow \text{mid} = 9 \Rightarrow 888 < 999$$

Element not found

Data size

if $n \leq 20$ Linear search is faster because sorting & binary search overhead are unnecessary.

$n > 20$

Binary search is preferred

If sorted use Binary search otherwise Linear search

Binary search is approximately 100 times faster.

```
#include <stdio.h>
```

```
int main()
```

```
{ int  
arr[] = { 66, 111, 212, 323, 432, 543, 643, 649, 777, 888 };
```

```
int key, low=0, high=9, mid;
```

```
printf("Enter key:");
```

```
scanf("%d", &key);
```

```
while (low <= high)
```

```
{ mid = (low + high) / 2;
```

```
if (arr[mid] == key)
```

```
{ printf("Found at position %d", mid+1);  
return 0;
```



```

}
else if (key < arr[mid])
    high = mid - 1;

```

```

else
    low = mid + 1;

```

```

}
printf("Element not found");
}

```

Complexity: Best = $O(1)$, worst = $O(\log n)$, space = $O(1)$

Initial list

NULL $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ 40 $\rightarrow$ 50 $\rightarrow$ 60 $\rightarrow$ NULL

Insert at position 3

Insert 25

Result: 10 $\rightarrow$ 20 $\rightarrow$ 25 $\rightarrow$ 30 $\rightarrow$ 40 $\rightarrow$ 50 $\rightarrow$ 60

Delete position 2

Delete: 20

Result: 10 $\rightarrow$ 25 $\rightarrow$ 30 $\rightarrow$ 40 $\rightarrow$ 50 $\rightarrow$ 60

Complexity: Insertion: $O(n)$, Deletion: $O(n)$

Array size = 10

Stack 1 $\rightarrow$ 0 1 2 3 4 5 6 7 8 9 + stack 2

stack 1 grows right to left, stack 2 grows right to left

Overflow occurs only when

$Top1 + 1 = Top2$

Advantages: Easy implementation

Disadvantages: Memory wastage, Early overflow

Both stacks share memory.

Advantages:

Better memory utilisation

Overflow only when array is completely full.

$$A = \left\lfloor \frac{C/D + (E + F * G) / H}{J} \right\rfloor$$

$$B \div A = B \div (E + F * G) \div (H \div J) \div J$$

a) $9384 | * +$

$8 | 4 = 2$

$3 \times 2 = 6$

$9 + 6 = 15$

Answer = 15

b) $62 + 63 | *$

$6 + 2 = 8$

$8 | 3 = 2$

$8 * 2 = 16$

$((a+b) + c * (d+e) + f) * (g+h)$

$* + + + ab * c + de f + gh$

$* + + + ab * c + de f + gh$ Postfix: $ab + cde + * + f + gh + *$

5. Size = 5, front = -1, rear = -1

Enqueue 20 20

Enqueue 10 20 10

30 20 10 30

Dequeue 10 30

Enqueue 80 10 30 80

Dequeue 30 80

Enqueue 90 30 80 90

Enqueue 100 30 80 90 100

Dequeue 80 90 100

Dequeue 90 100

Enqueue 60 90 100 60

Enqueue 90 90 100 60 90

Enqueue Complexity
 $O(1)$

Dequeue $O(1)$

Display $O(n)$